

Version Management

Admin Operations and Configuration Manual

Manual 24 of 25
Route: /admin-version-management
Group: Tools
Generated: May 10, 2026

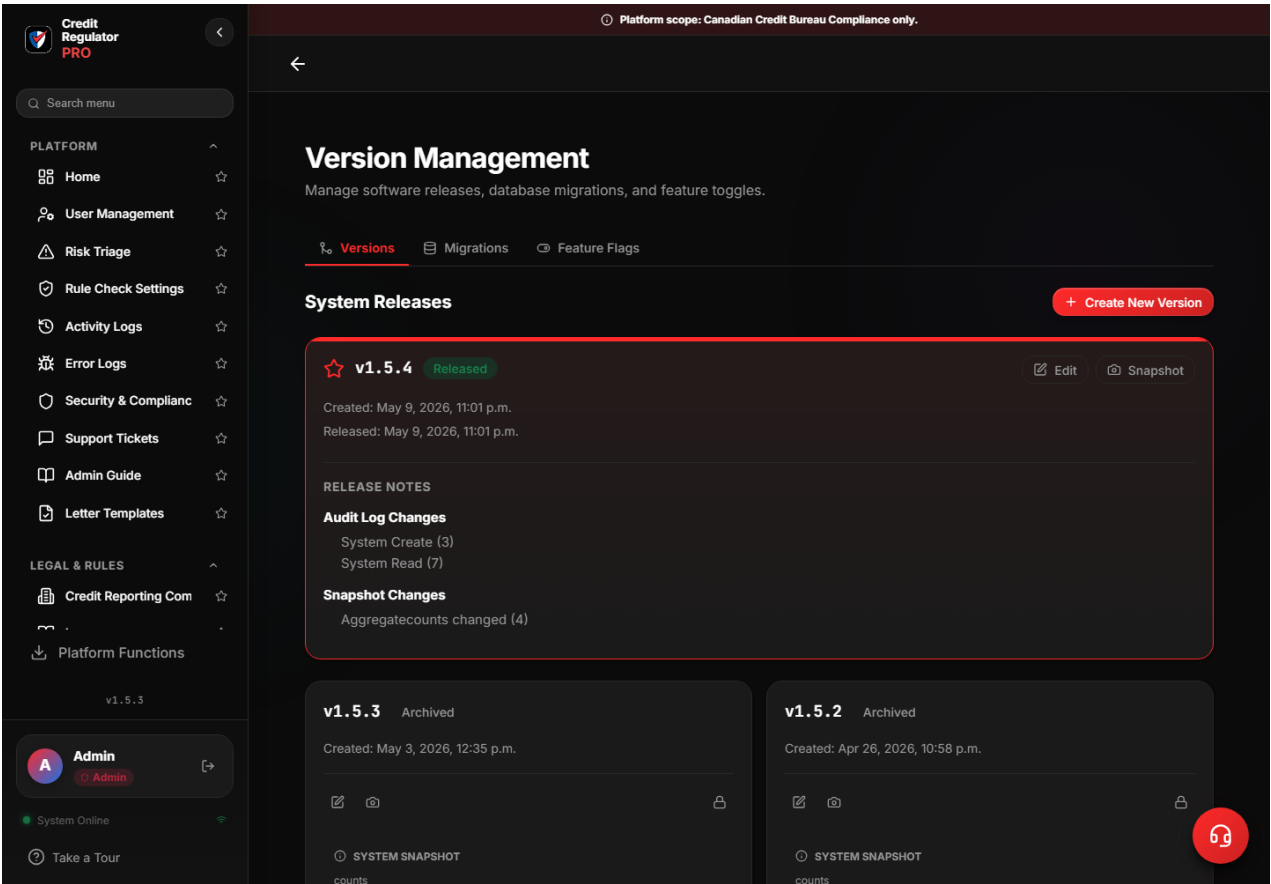
1. Section Overview

Version Management tracks release records, migration metadata, and feature flags. It helps admins understand release state and staged feature availability.

Use this section to review release information, mark migration metadata after verification, and enable or disable feature flags under approved change control.

2. Current Admin Screenshot

Screenshot source: localhost development admin environment with test data. Test data is permitted for these manuals and should still be treated as internal.



The screenshot shows version, migration, and feature-flag management tabs.

3. Access and Operating Rules

- Sign in with an account that has admin access. Support users may see some reference sections, but admin configuration and review tools require admin role.
- Use the intended environment. Local admin browser testing uses `http://localhost:5175`. The backend port, `http://localhost:3333`, is API-only.
- Treat screenshots, exports, logs, report text, and record IDs as internal test-environment material. Do not paste them into public channels.
- Confirm the page route, active filters, selected user, selected report, and selected environment before any save, delete, reset, dismissal, or finalize action.
- When an action affects customer-facing findings, packets, pricing, feature flags, or parser truth, record the reason and verify the downstream record after refresh.
- Migration buttons are metadata controls. They do not run database migrations.
- Feature flags can expose or hide user/admin functionality.
- Release notes should reflect actual deployed behavior and known risks.

4. Screen Anatomy and Controls

Area	What it means	Admin procedure
Versions	Lists release/version records and notes.	Use for release context and operational history.
Migrations	Shows migration metadata and mark-applied/rolled-back controls.	These buttons mark metadata; they do not execute SQL.
Feature Flags	Controls staged feature availability.	Change flags only with approval and rollback plan.

5. Step-by-Step Instructions

Review current release version

Use before support diagnosis, deployment signoff, or staging validation.

Procedure

1. Open Version Management and select Versions.
2. Identify current version, date, notes, and status.
3. Compare release notes to the workflow under review.
4. Open Activity Logs if the release metadata was recently changed.
5. Document version context in support or validation notes.

Verification

- Version context is known before diagnosing release-sensitive behavior.

Mark migration metadata

Use only after an actual migration has already been verified by the deployment process.

Procedure

1. Open Migrations and find the migration record.
2. Confirm the migration was applied or rolled back outside this UI.
3. Read the current metadata status.
4. Click Mark Applied or Mark Rolled Back according to verified state.
5. Refresh and confirm metadata state persists.
6. Record verification source and timestamp.

Decision options

- Metadata matches verified migration state.
- Activity Logs show the metadata change.

Verification

- Mark Applied: use after confirmed successful migration.
- Mark Rolled Back: use after confirmed rollback.
- Do not use: when you are trying to execute SQL or repair schema.

6. Common Tasks With Drilldown Examples

Toggle a feature flag

Example: Enable a staged admin tool for approved testing.

Drilldown procedure

1. Open Feature Flags.
2. Find the exact flag by name and description.
3. Confirm audience, environment, approval, and rollback plan.
4. Change the flag state and save.
5. Refresh and verify state persists.
6. Open the affected route or workflow and confirm behavior matches expectation.
7. Record the flag state change in the release/admin note.

Best practices for this task

- Use flags for approved staged access, not ad hoc experiments on users.
- Know how to disable the flag before enabling it.

Done when

- Flag state persists and affected behavior matches expected access.

Create or update release notes

Example: A staging repair changed admin sidebar discoverability.

Drilldown procedure

1. Open Versions and select the current or target release record.
2. Add concise notes describing changed admin behavior, checks run, and residual risks.
3. Avoid secrets, credentials, or private deployment details.
4. Save and refresh.
5. Confirm notes are visible to admins who need release context.

Best practices for this task

- Release notes should summarize operational impact, not implementation noise.

Done when

- Notes persist and accurately describe the release.

7. Configuration and Change-Control Scope

- Version records are operational metadata.
- Migration status metadata affects admin visibility into deployment state.
- Feature flags can change application behavior for selected audiences.

8. Common Errors

Problem	Likely cause	Admin response
Access denied or redirected to login	The session expired, the role is not admin, or the wrong environment is open.	Sign back in, confirm admin role, and reopen the route from the sidebar instead of using an old bookmark.
Expected record is missing	Search terms, tabs, filters, pagination, or environment selection are hiding the record.	Clear filters, search by exact ID or email, verify the environment, and compare Activity Logs when available.
Save appears to do nothing	Required fields are missing, there are no dirty changes, or the API returned a validation error.	Read the inline validation text, correct one field at a time, save again, refresh, and check Error Logs if the state still does not persist.
version management state conflicts with another page	The source record was changed elsewhere or the current page is stale.	Refresh the page, reopen the source record, and use timestamps or record IDs to decide which state is authoritative.

9. Troubleshooting

Check	Why it matters	How to proceed
Start with a refresh	A refresh confirms whether the version management view is stale or whether the	Refresh once, then repeat the exact search or selection. Do not repeat destructive actions until the

Check	Why it matters	How to proceed
	backend rejected the action.	latest state is visible.
Check Activity Logs	Successful admin actions should leave an audit trail for sensitive workflows.	Search by actor, affected user, entity ID, route, or timestamp. Use the log entry as confirmation, not just the toast message.
Check Error Logs	Repeated UI failures usually have matching server or API errors.	Search around the action time. Capture the route, status, error summary, and request context for developer follow-up.
Escalate with evidence	Developer review is faster when the exact record and reproduction path are known.	Provide environment, route, selected filters, record ID, expected outcome, actual outcome, screenshot, and timestamp.

10. Best Practices

- Do not confuse migration metadata with migration execution.
- Change feature flags only with approval.
- Keep rollback paths clear.
- Record release context for support-sensitive changes.

11. Completion Checklist

- The version management page was reopened or refreshed after the change.
- The expected saved state remained visible after refresh.
- Related logs, downstream records, or generated artifacts agree with the visible page state.
- Any customer-facing wording, finding status, packet state, or support note was reviewed for factual and neutral language.
- Any unresolved risk, failed test, suspicious log, or unclear source evidence was escalated with record IDs and screenshots.

12. Related Admin Areas

- Activity Logs
- Error Logs
- Rule Check Settings
- Admin Guide
- Lifecycle Testing